Performance Analysis of Redis and MongoDB Caching for Web Applications

Harshit Khandelwal
Bachelor In Computer Application
JECRC UNIVERSITY, Jaipur, Rajasthan, India
khandelwalharshit2024@gmail.com

***Abstract*—In modern large-scale web application development, minimizing data retrieval latency and server load is critical to delivering an optimal user experience under heavy, concurrent traffic. Web caching addresses these scalability challenges by storing frequently accessed data in high-speed, intermediate storage layers. This paper presents an empirical comparative performance analysis of two prominent database technologies evaluated as caching layers: Redis, an in-memory key-value data structure store, and MongoDB, a document-oriented NoSQL database. Utilizing a custom Node.js benchmarking framework, we evaluate both systems across fundamental Create, Read, Update, and Delete (CRUD) operations under 1,000 operation iterations using pipelined and bulk execution mechanisms. We measure execution latency in milliseconds (ms) and throughput in operations per second (Ops/Sec). The empirical findings reveal that Redis achieves sub-millisecond read latency (0.5133 ms) owing to its RAM-first hash-map architecture—outperforming MongoDB (10.0821 ms) by approximately $20\times$ for read-heavy key-value caching workloads. Conversely, MongoDB demonstrates exceptional performance during collection-level bulk eviction (0.1180 ms, 8,487 ops/sec). This paper provides system architects with actionable guidelines for selecting the optimal caching technology based on specific application workload patterns and data complexity requirements.**

***Index Terms*—Redis, MongoDB, Web Caching, Database Performance, CRUD Latency, NoSQL, Throughput, Memory Management, System Architecture.**

I. Introduction

A. Motivation and Background

In contemporary web application engineering, delivery speed, system availability, and low latency are foundational prerequisites for maintaining user engagement. As online platforms scale to support thousands of simultaneous concurrent users, traditional relational database management systems (RDBMS) such as MySQL and PostgreSQL frequently encounter severe throughput bottlenecks [1]. Traditional disk-bound databases rely on physical disk storage (HDD/SSD) and complex SQL B-tree indexing mechanisms, which incur substantial disk I/O seek times and CPU overhead during high-concurrency traffic spikes.

To bypass these hardware constraints, modern software architectures implement web caching layers [2]. Caching relies on the locality-of-reference principle, positioning a high-speed, temporary data storage layer between the application server tier and the primary database tier. By intercepting incoming read requests and returning pre-computed or frequently accessed data directly from fast Random-Access Memory (RAM), caching eliminates redundant database queries, lowers server CPU utilization, and reduces user page-load latencies.

The fundamental architectural data flow in a cached web application is illustrated as follows:

$$\text{Client User} \longrightarrow \text{Node.js API} \longrightarrow \text{RAM Cache} \longrightarrow \text{Primary DB (Disk)}$$

B. Problem Statement

When designing a high-performance web caching infrastructure, software engineers face a critical decision: Which storage engine provides optimal throughput and minimal latency for cached web objects—a dedicated in-memory key-value store (Redis) or a document-oriented NoSQL database (MongoDB)?

This research evaluates two prominent NoSQL systems under identical execution conditions:

1) **Redis** [3]: An open-source, in-memory data store engineered for sub-millisecond key-value retrieval. It retains the entire working dataset in RAM, mapping simple string keys to structured values:

   `"user:101"` $\longrightarrow$ `"Harshit Khandelwal"`

2) **MongoDB** [5]: A document-oriented NoSQL database that stores data in flexible Binary JSON (BSON) format, supporting complex nested fields, dynamic schemas, and secondary index structures:

```
{
  "userId": "user_102938",
  "name": "Harshit Khandelwal",
  "role": "Developer",
  "skills": ["React", "Node.js"]
}
```

While MongoDB is traditionally deployed as a persistent primary database and Redis as a dedicated cache, modern engineering teams frequently evaluate MongoDB as a unified caching layer for complex structured objects to minimize infrastructure complexity. This paper directly addresses this choice by conducting an empirical head-to-head comparison across all standard CRUD operations.

C. Research Questions

This investigation is guided by three core research questions:

- **RQ1**: What is the latency and throughput trade-off between Redis and MongoDB across Create, Read, Update, and Delete operations?
- **RQ2**: By what exact quantitative factor does Redis's pure RAM architecture outperform MongoDB during read-intensive caching workloads?
- **RQ3**: Are there specific caching operations where MongoDB's document architecture offers speed or operational advantages over Redis?

D. Main Contributions

The key contributions of this paper are summarized as follows:

- **Empirical Benchmarking Suite**: Development of an asynchronous Node.js benchmarking framework utilizing pipelined Redis requests and MongoDB bulk operations across 1,000 iterations.
- **Quantitative Performance Data**: Empirical proof demonstrating that Redis achieves sub-millisecond read latency (0.5133 ms) and 1,948 ops/sec, outperforming MongoDB (10.0821 ms) by approximately $20\times$.
- **Bulk Eviction Discovery**: Identification of MongoDB's superior performance during collection-level bulk cache eviction (0.1180 ms, 8,487 ops/sec).
- **Architectural Guidelines**: A comprehensive decision framework helping engineers select database caching solutions based on data complexity and workload patterns.

E. Paper Structure

The remainder of this paper is structured as follows: Section II surveys related work. Section III details the methodology. Section IV describes the experimental setup. Section V presents empirical results. Section VI outlines limitations and future work, and Section VII concludes the paper.

II. Related Work

A. In-Memory Key-Value Caching

In-memory caching architectures prioritize execution speed by retaining data exclusively in RAM. Studies by Li et al. [4] demonstrated that in-memory key-value stores like Redis reduce primary database query loads by over 80% in distributed web systems. Redis utilizes a single-threaded event loop built on non-blocking I/O multiplexing, eliminating CPU thread context-switching overhead and locking contention.

B. Document-Oriented NoSQL Systems

Document-oriented NoSQL databases were introduced to address the schema rigidity of traditional RDBMS. Cooper et al. [6] introduced YCSB to evaluate NoSQL storage engines. Their findings established that while document databases introduce slight CPU overhead due to BSON parsing, they excel at handling dynamic schema evolution. MongoDB's WiredTiger storage engine incorporates an internal RAM cache to accelerate document access.

C. Comparative Caching Taxonomy

Table I compares the architectural features of Redis and MongoDB across fundamental database design parameters.

TABLE I
ARCHITECTURAL FEATURE MATRIX: REDIS VS. MONGODB

Feature Parameter	Redis Store	MongoDB Store
Primary Data Model	Key-Value Pair	BSON Document
Primary Storage Medium	Pure RAM	Disk + RAM Cache
Threading Model	Single-Threaded	Multi-Threaded
Secondary Indexing	Limited	Full Indexing
Query Language	Commands/RESP	MQL / Aggregation
Persistence Options	RDB / AOF	Write-Ahead Logs
Default Port	6379	27017

D. Research Gap Identification

Existing literature overwhelmingly focuses on Redis as a standalone cache or MongoDB as a persistent database. Few studies evaluate MongoDB configured explicitly as an intermediate caching layer against Redis under identical Node.js asynchronous execution loops.

III. Proposed Methodology

A. System Architecture Overview

The benchmarking environment comprises an asynchronous Node.js client application communicating with Redis Cloud and MongoDB Atlas instances. The application issues high-speed CRUD requests and records high-resolution timestamps using `performance.now()`.

$$\text{Node.js Client} \longrightarrow \text{Execution Loop} \longrightarrow \begin{cases} \text{Redis (RESP)} \\ \text{MongoDB (BSON)} \end{cases}$$

B. Pipelining and Bulk Processing Mechanics

To isolate true database processing speeds:

- **Redis Pipelining**: We utilize `redis.pipeline()` to pack 1,000 operations into a single TCP frame.
- **MongoDB Bulk Execution**: We utilize `insertMany()`, concurrent `Promise.all()` queries, and collection-level `deleteMany({})` commands.

C. Benchmark Algorithms

Listing 1 details the Redis benchmark algorithm and Listing 2 details the MongoDB execution module.

```
// Input: ITERATIONS = 1000, sampleData
// Output: Execution Latency (ms)
1. Connect to Redis Cloud Instance
2. Flush previous database keys (FLUSHDB)
3. Initialize Redis Pipeline object
4. For i = 0 to ITERATIONS - 1:
     Pipeline.set("user:" + i, sampleData)
5. Execute Pipeline.exec() and measure time
6. Record CREATE Latency
7. Repeat step 3-5 for GET, SET, and DEL commands
8. Disconnect Redis client
```

Listing 1. Redis Pipelined Benchmark Execution

```
// Input: ITERATIONS = 1000, sampleData
// Output: Execution Latency (ms)
1. Connect to MongoDB Atlas Cluster
2. Drop existing collection data
3. Construct array of 1000 BSON documents
4. Measure start time
5. Execute collection.insertMany(docs)
6. Record CREATE Latency
7. Execute collection.findOne() via Promise.all()
8. Record READ Latency
9. Execute collection.deleteMany({}) for bulk purge
10. Disconnect MongoDB client
```

Listing 2. MongoDB Bulk Benchmark Execution

IV. Experimental Setup

A. Hardware and Environment Specifications

All benchmarks were conducted on a workstation configured with the technical parameters detailed in Table II.

TABLE II
Experimental Hardware & Software Setup

Component	Specification
Operating System	Microsoft Windows 10
Processor	Multi-Core Intel
Runtime Environment	Node.js v18.x (V8 Engine)
High-Res Timer	`performance.now()` (Sub-ms precision)
Redis Driver	ioredis v5.x
MongoDB Driver	mongodb native v6.x
Redis Instance	Managed Redis Cloud (In-Memory)
MongoDB Instance	Managed MongoDB Atlas (WiredTiger)
Test Iterations (N)	1,000 Operations per CRUD step

B. Standardized JSON Workload Payload

To simulate realistic web application session caching, an identical JSON payload was used across all operations:

```
{
  "userId": "user_102938",
  "name": "Harshit Khandelwal",
  "role": "Developer",
  "sessionToken": "xyz_abc_987654321",
  "lastAccess": 1727641800
}
```

C. Mathematical Performance Metrics

Performance evaluation relies on two standard metrics:

1) **Average Latency (ms)**:

$$\text{Average Latency (ms)} = \frac{\text{Total Execution Time (ms)}}{N} \tag{1}$$

2) **Throughput (Ops/Sec)**:

$$\text{Throughput (Ops/Sec)} = \left(\frac{N}{\text{Total Execution Time (ms)}} \right) \times 1000 \tag{2}$$

V. Results and Discussions

The empirical performance data recorded across 1,000 operations are presented in Table III and Table IV.

TABLE III
Average Latency Comparison (Lower is Better)

Operation	Redis Latency	MongoDB Latency	Faster System
Create	1.2587 ms	3.3682 ms	**Redis**
Read	**0.5133 ms**	10.0821 ms	**Redis** (20×)
Update	0.6205 ms	8.9554 ms	**Redis** (14×)
Delete	0.3114 ms	**0.1180 ms**	**MongoDB**

TABLE IV
Throughput Comparison (Higher is Better)

Operation	Redis (Ops/Sec)	MongoDB (Ops/Sec)	Higher Speed
Create	794	297	**Redis**
Read	**1948**	99	**Redis**
Update	1612	112	**Redis**
Delete	3211	**8474**	**MongoDB**

A. Read Performance Analysis (Primary Finding)

Data retrieval is the single most critical operation in web caching. As demonstrated in Table III, Redis recorded an average read latency of 0.5133 ms (1,948 ops/sec) compared to MongoDB's 10.0821 ms (99 ops/sec), representing a $20\times$ performance advantage. Redis operates as a pure in-memory hash map with $O(1)$ RAM access without BSON parsing or index traversal. In contrast, MongoDB must locate documents within its B-tree index, traverse WiredTiger cache buffers, and deserialize BSON fields.

B. Create and Update Workload Analysis

- **Create Operation**: Redis achieved 1.2587 ms (794 ops/sec) vs MongoDB's 3.3682 ms (297 ops/sec)—a $2.67\times$ advantage.
- **Update Operation**: Redis achieved 0.6205 ms (1,612 ops/sec) vs MongoDB's 8.9554 ms (112 ops/sec)—a $14\times$ advantage.

Redis modifies string values directly in RAM. MongoDB must execute document update routines, modify index nodes, and record write-ahead log (journaling) entries.

C. Bulk Delete Performance Analysis (Secondary Finding)

An intriguing result emerged during collection-level cache clearing: MongoDB recorded a bulk deletion latency of 0.1180 ms (8,487 ops/sec) compared to Redis's 0.3114 ms (3,211 ops/sec). MongoDB outperformed Redis because `deleteMany({})` clears collection metadata pointers at the storage engine level in a single operation, whereas Redis pipelined individual `DEL` keys.

D. Architectural Trade-Off Summary

Table V provides a decision matrix summarizing optimal database selection criteria.

TABLE V
CACHING TECHNOLOGY SELECTION MATRIX FOR WEB APPLICATION ARCHITECTS

Application Requirement	Recommended	Primary Rationale
Sub-millisecond Read Speed	Redis	RAM Lookup via Single-Threaded Loop
Simple Key-Value Session Data	Redis	Minimal Memory Footprint
High-Frequency Update Loops	Redis	In-Memory Mutation Without Journaling
Complex Nested Query Filters	MongoDB	Secondary Indexing & BSON Traversal
Bulk Collection Cache Purging	MongoDB	Fast Metadata Invalidation (0.1180 ms)
Unified DB + Cache Layer	MongoDB	Eliminates Separate Infrastructure

VI. LIMITATIONS AND FUTURE WORK

A. Research Limitations

1) **RAM Cost Scaling**: Redis holds datasets entirely in RAM, which incurs higher infrastructure costs for multi-gigabyte caches.
2) **Data Volatility Risks**: Unscheduled power failures can cause data loss in Redis unless disk persistence (RDB/AOF) is enabled.
3) **Single-Node Scope**: Experiments were conducted on single-node cloud instances.

B. Future Research Directions

Future work will focus on benchmarking distributed multi-node clusters (Redis Cluster vs MongoDB Sharded Cluster), conducting high-concurrency load testing ($10,000+$ virtual users), and profiling RAM/CPU core utilization under stress.

VII. CONCLUSION

This paper presented an empirical performance analysis of Redis and MongoDB as caching layers. Redis is the optimal choice for simple, read-heavy caching, achieving sub-millisecond read latency (0.5133 ms) and $20\times$ higher throughput than MongoDB. However, MongoDB offers flexibility for complex structured objects and exhibits superior performance during bulk collection eviction (0.1180 ms).

REFERENCES

[1] R. Cattell, "Scalable SQL and NoSQL data stores," *ACM SIGMOD Rec.*, vol. 39, no. 4, pp. 12–27, 2011.
[2] A. Vacali and G. Pallis, "Web caching and content delivery networks," *IEEE Internet Comput.*, vol. 6, no. 6, pp. 24–31, 2002.
[3] Redis Ltd., "Redis Documentation: In-Memory Data Structures Store," 2024. [Online]. Available: https://redis.io/docs/
[4] Z. Li, C. Wilson, and H. Zhao, "Evaluating in-memory key-value store performance in modern cloud applications," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 880–892, 2021.
[5] MongoDB Inc., "MongoDB Manual: Document Oriented Database Architecture," 2024. [Online]. Available: https://www.mongodb.com/docs/
[6] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proc. 1st ACM Symp. Cloud Comput. (SoCC)*, 2010, pp. 143–154.
[7] R. Nishtala *et al.*, "Scaling Memcache at Facebook," in *Proc. 10th USENIX Conf. Networked Syst. Des. Impl. (NSDI)*, 2013, pp. 385–398.
[8] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.
[9] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to build high-performance network programs," *IEEE Internet Comput.*, vol. 14, no. 6, pp. 80–83, 2010.
[10] M. Armbrust *et al.*, "A view of cloud computing," *Commun. ACM*, vol. 53, no. 4, pp. 50–58, 2010.